

Aayuvridhi Healthcare System

Research and Technology Notes

1. Project summary

Aayuvridhi Healthcare System is a Python command-line application. A user types a disease, symptom, medical condition, body problem or medicine name. The system searches a local database first and shows the answer in two styles:

- **Patient view:** plain-language explanation, common signs, home care, warning signs and which doctor to consult.
- **AI Doctor view:** clinical summary, possible causes, suggested investigations, red flags and the relevant specialist.

If nothing is found locally, the program falls back to searching online medical sources (NIH MedlinePlus web service).

2. What was used

Area	Tool / library	Purpose
Language	Python 3	Whole application
Editor	Visual Studio Code	Development
Local storage	SQLite (sqlite3, built into Python)	Stores the disease database (data/diseases.db) and the healthcare database (data/healthcare.db)
Data generation	pandas	generate_large_dataset.py builds large CSV datasets
Online lookup	requests	Calls the MedlinePlus search service
Web page reading	BeautifulSoup (bs4)	Reads and cleans text from web results
Text handling	re, difflib, csv, os (standard library)	Tokenising queries, typo tolerance, CSV import, file paths
Version control	Git and GitHub	Source code hosting and submission

3. Project files

File	Role
src/query.py	Main search program. Tries the local disease database first, then online sources.
src/disease_db.py	Disease knowledge base: creates and fills the SQLite table, searches it, prints Patient and AI Doctor views.
src/database.py	Database handling for the healthcare data.
src/cleaner.py	Cleans and prepares text and data.
src/build_datasets.py	Builds datasets used by the system.
src/generate_large_dataset.py	Generates large modern-medicine and Ayurvedic-ingredient CSV files (data/processed/).
src/main.py	Program entry point.

4. Research about the data

4.1 Disease database (disease_db.py)

The disease table was written by hand from general, widely published medical knowledge and currently holds 16 common conditions: fever, common cold, influenza, dengue, malaria, typhoid, gastroenteritis, migraine, hypertension, type 2 diabetes, asthma, urinary tract infection, dental abscess, gingivitis, pneumonia and anemia.

Each record has these fields:

Field	Meaning
name	Standard disease or symptom name
aliases	Other names, including everyday and local words (for example "bukhar" for fever)
category	Body system or type (respiratory, dental, infectious, and so on)
symptoms	Common signs a patient notices
description	Short plain explanation
causes	Common causes
home_care	Safe self-care measures
red_flags	Warning signs that need urgent medical attention
doctor_type	Which specialist to see
tests	Investigations a doctor may order

The database is designed to grow. The `import_csv()` function in `disease_db.py` loads any CSV that has a `name` column, so a larger disease list can be added without changing the code.

4.2 Generated datasets (generate_large_dataset.py)

This script produces two CSV files by looping over medical categories and their symptoms and creating several variants of each record:

- `data/processed/modern_medicines.csv`
- `data/processed/ayurvedic_ingredients.csv`

These records are **programmatically generated for testing and demonstration**. They are not verified clinical records.

4.3 Online source

When the local database has no match, `query.py` sends the search to the NIH MedlinePlus web search service (wsearch.nlm.nih.gov). MedlinePlus is a public health-information service run by the U.S. National Library of Medicine.

5. How the search works

- 1 The query is lowercased and common filler words are removed ("in", "my", "have", and so on).
- 2 Words are shortened to a simple root form ("swellings" becomes "swelling").
- 3 Synonyms are added. For example, "bukhar" and "temperature" map to fever, and "tooth" maps to dental.

- 4 Every disease is scored: a match in the name counts most, then aliases, then symptoms, category and description.
- 5 An exact match on a name or alias gets a bonus, so searching "fever" ranks the Fever entry above Dengue and Typhoid.
- 6 If nothing matches, the system tries close spellings using difflib, which handles typos such as "feveer".
- 7 The top results are printed in Patient view and AI Doctor view.

Example: searching *swelling in teeth* returns Dental Abscess and Gingivitis.

6. Limitations

- The disease database covers common conditions only and is not a complete medical database.
- The "AI Doctor" view is rule-based, built from the stored data. It does not use a live AI model and cannot examine a patient.
- Generated datasets are synthetic and must not be used for real treatment decisions.
- Medicine dosage is deliberately not included.
- Content should be reviewed by a qualified medical professional before real-world use.

7. Safety and ethics

- Every result ends with a disclaimer: the information is general and not a diagnosis.
- Red-flag symptoms are shown so users know when to seek urgent care.
- The program does not store or collect personal patient information.

8. References for verification

The condition summaries follow standard public health guidance. These are the recommended sources to check the content against:

- NIH MedlinePlus: <https://medlineplus.gov>
- World Health Organization fact sheets: <https://www.who.int/news-room/fact-sheets>
- U.S. CDC health topics: <https://www.cdc.gov>
- NHS health A to Z: <https://www.nhs.uk/conditions>

9. Future work

- Add more diseases by importing a larger CSV.
- Connect the AI Doctor view to a language model for free-text questions.
- Add a graphical or web interface.
- Add multi-language support (Hindi and others).
- Add doctor review and source links for each record.